MICROSOFT · OPEN SOURCE

APM

A dependency manager for AI agents. **Portable** by manifest, **secure** by default, **governed** by policy — and how it pairs with **AgentRC**.

For: Developers & Architects **Duration:** 30 min + Q&A **Speaker:** Soham Dasgupta · Sr. Partner Solution Architect

THE PROBLEM

Every agent. Every dev. Every machine. Different setup.

What teams ship today

- A README that says "install these extensions"
- A copy-pasted `copilot-instructions.md`
- Ad-hoc MCP server configs in three different files
- Skills, prompts, and rules duplicated per agent (Copilot today, maybe Claude or Cursor tomorrow)
- No version pinning. No integrity. No reproducibility.

The result

- "It works on my agent" 🤷
- Drift between developers
- Silent context rot as the repo evolves
- No way to ship agent context as a versioned artifact
- No security boundary around prompts — which *are* programs for an LLM

THE IDEA

What if agent context had a package.json?

One manifest. One install. Every agent, configured.

```yaml
# apm.yml — ships with your repo
name: your-project
version: 1.0.0
dependencies:
  apm:
    # Skills, prompts, agents, plugins — from any git repo, version-pinned
    - github/awesome-copilot/skills/review-and-refactor
    - github/awesome-copilot/plugins/context-engineering#v2.1
    - microsoft/apm-sample-package#v1.0.0
  mcp:
    # MCP servers governed by the same manifest (registry entries are bare strings)
    - io.github.github/github-mcp-server
```

```
$ git clone <repo> && cd <repo>
$ apm install
→ resolving 14 packages…
→ writing .github/copilot-instructions.md
→ writing .github/prompts/, .github/instructions/
→ writing AGENTS.md (read by Claude, Cursor, Codex, …)
✓ every agent in the repo is configured
```

THE THREE PROMISES

What APM guarantees

1. Portable by manifest

One `apm.yml` describes every primitive — instructions, skills, prompts, agents, hooks, plugins, MCP servers. `apm install` reproduces the same setup across every harness on every machine. `apm.lock.yaml` pins the resolved tree.

2. Secure by default

Agent context is executable in effect — a prompt is a program for an LLM. Every install scans for hidden Unicode, pins content hashes, and gates transitive MCP servers behind trust prompts.

3. Governed by policy

`apm-policy.yml` is enforced at install time, including transitive MCP. Tighten-only inheritance flows enterprise → org → repo.

PRIMITIVES

What APM packages can contain

INSTRUCTIONS

Repo conventions, architecture, do/don't rules. Rendered into `.github/copilot-instructions.md` (and `AGENTS.md` for non-Copilot harnesses).

PROMPTS

Slash-command prompts. Deployed to `.github/prompts/` for Copilot.

AGENTS

Single-purpose agent files (e.g. `api-architect.agent.md`). Deployed to `.github/agents/`.

SKILLS

Reusable capabilities in the Agent Skills format (`SKILL.md`). Drop-in from `anthropics/skills` or any repo.

HOOKS & COMMANDS

Lifecycle hooks and CLI commands the agent can invoke. Governed by policy.

MCP SERVERS

Declared in the same manifest. Policy-checked and trust-gated before they touch disk.

Eight primitive types in total (instructions, prompts, agents, skills, hooks, commands, plugins, MCP). Plugins are a bundle of the others — see slide 9.

CLI WALKTHROUGH

The five commands you'll actually use

Pick a flow: apm install | apm install <pkg> | apm compile -t copilot | apm audit | apm pack

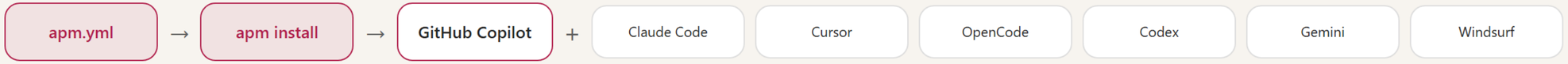
```
$ apm install
Resolving apm.yml…
  → github/awesome-copilot/skills/review-and-refactor  ok
  → github/awesome-copilot/plugins/context-engineering@v2.1  ok
  → microsoft/apm-sample-package@v1.0.0  ok
Scanning for hidden Unicode…  clean
Verifying content hashes against apm.lock.yaml…  match
Deploying primitives → .github/, .vscode/, AGENTS.md
✓ 14 packages installed · 0 warnings
```

▶ Replay Animated terminal — illustrative output

COMPILE TARGETS

Copilot-first. Portable to the rest.

The same packages render to whichever harness your team uses.



ZERO-CONFIG FOR COPILOT

`apm install` writes `.github/copilot-instructions.md`, `.github/prompts/`, `.github/instructions/`, and `.vscode/mcp.json`. VS Code and GitHub Copilot read all of these automatically — no settings to flip. APM itself dogfoods the Copilot target.

The same install also emits `AGENTS.md` in the repo root — the open agents.md standard that Claude Code, Cursor, OpenCode, Codex, Gemini, and Windsurf all read. **You don't have to use them; if a teammate does, their agent is already configured.**

INTERACTIVE DEMO

Assemble an `agents.md` from APM packages

Pick the packages your project needs. See what gets generated.

Available packages

- review-and-refactor
- context-engineering
- agent api-architect.agent
- instructions security-rules (GitLab)
- github-mcp-server
- mcp playwright-mcp

Generated `apm.yml`

```
name: my-project
version: 1.0.0
dependencies:
  apm:
    - github/awesome-copilot/skills/review-and-refactor
    - github/awesome-copilot/plugins/context-engineering#v2.1
  mcp:
    - io.github.github/github-mcp-server
```

Generated `AGENTS.md` 16 lines

```
$ apm install   # deploys primitives + writes AGENTS.md
```

```
# AGENTS.md
Generated by `apm compile` from apm.yml.

## Review & refactor conventions
- Review for security, perf, naming, and missing tests.
- Refactor in small, behavior-preserving steps.
- Prefer composition; keep modules cohesive.

## Context engineering
- Keep instructions task-scoped; avoid kitchen-sink files.
- Document the build/test/lint commands at the top.
- Refresh context when architecture decisions change.

## MCP: github-mcp-server
- Available tools: list_issues, create_pr, review_pr, search_code.
- Scope: the repo this manifest belongs to.
```

`apm install` deploys primitives (skills, prompts, agents, MCP) *and* writes the rendered `AGENTS.md`. For Copilot, that includes `.github/copilot-instructions.md`, `.github/prompts/`, `.github/instructions/`, and `.vscode/mcp.json`. Use `apm compile` separately to regenerate just the rendered instructions files.

PLUGINS & BUNDLES

Why plugins? Because a real capability is more than one file.

A skill is one primitive. A workflow needs several primitives bundled together.

Concrete example

A **Code Review** plugin your platform team wants every repo to have:

- 📜 **Instructions** — review rubric (security, perf, naming)
- 🤖 **Agent** — `code-reviewer.agent.md` activated by `/review`
- 🧰 **Skill** — "summarise diff" + "find missing tests"
- 🔌 **MCP server** — `github-mcp-server` for posting PR comments

Without a bundle, every dev wires four files by hand. **With `apm install acme/code-review`, all four primitives drop into the right places under `.github/` and `.vscode/` in one command — pinned in `apm.lock.yaml`.**

Marketplaces

Curated registries of packages. `apm marketplace add` + one `apm install`, lockfile-pinned. `apm pack` emits `marketplace.json` alongside the bundle when your manifest declares one.

```
# package layout — primitives live in .apm/, not in apm.yml
acme/code-review/
├── apm.yml                        # name, version, dependencies
├── .apm/
│   ├── instructions/rubric.instructions.md
│   ├── agents/code-reviewer.agent.md
│   ├── skills/summarise-diff/SKILL.md
│   └── skills/find-missing-tests/SKILL.md
└── README.md

# apm.yml declares only deps, not the primitives themselves
name: code-review
version: 1.2.0
dependencies:
  apm:
    - acme/shared-style-guide#v3   # transitive dep
  mcp:
    - io.github.github/github-mcp-server
```

```
# consume it from any repo
$ apm install acme/code-review#v1.2.0

# or pack a single-file bundle for offline / air-gapped delivery
$ apm pack # → build/code-review/ (plugin.json + agents/ + skills/ + commands/ + hooks/ + apm.
$ apm pack --archive # → build/code-review-1.2.0.tar.gz
```

SECURITY

Treat prompts like the programs they are

UNICODE SCANNING

Every install blocks hidden bidi / zero-width characters that can hijack agent behavior.

LOCKFILE INTEGRITY

`apm.lock.yaml` records resolved sources and content hashes — full provenance, drift-proof installs. Bundles embed it too, so `apm install <bundle>` rehashes every file before writing.

MCP TRUST GATING

Transitive MCP servers don't get installed unless explicitly declared or trusted.

APM AUDIT

On-demand scan for compromised packages, hidden chars, lockfile drift. CI-friendly.

DRIFT DETECTION

Detects when generated files diverge from what the manifest says — fail the build instead of shipping silently.

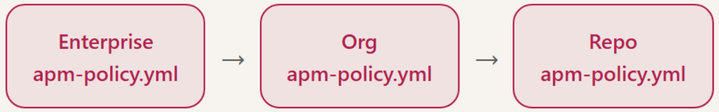
CI/CD READY

Official `microsoft/apm-action` for GitHub Actions. Plug it into PR checks.

GOVERNANCE

One policy file. Tighten-only inheritance.

How it flows



Each level can only *tighten* the policy, never loosen it. Enforced at `apm install` time — including transitive dependencies and MCP servers.

```yaml
# /.github/apm-policy.yml — auto-discovered for every repo in the org
name: "Contoso Engineering Policy"
version: "1.0.0"
enforcement: block          # warn | block | off

dependencies:
  allow:
    - "contoso/**"
    - "microsoft/*"
  deny:
    - "untrusted-org/**"
  require:
    - contoso/agent-standards     # every repo must pull this
  max_depth: 5                  # cap transitive depth
  require_pinned_constraint: true   # ban floating refs

mcp:
  allow:
    - "io.github.github/*"
  transport:
    allow: [stdio, streamable-http]
  self_defined: warn           # ad-hoc MCP entries warn

compilation:
  target:
    allow: [copilot]             # copilot is an alias for vscode
```

Wire `apm audit --ci --policy org` into PR checks — produces SARIF, renders inline on the PR diff via GitHub Code Scanning. Note: package signing is not yet implemented; lockfile presence is enforced as a baseline audit check (`lockfile-exists`), not a policy field.

AGENTRC — PART 1 OF 2 · EXPERIMENTAL

AgentRC: context engineering, automated

APM *distributes* agent context. AgentRC *generates* and *maintains* it. (repo is marked experimental — pin a commit if you adopt it)

What

A CLI + VS Code extension that reads your codebase and generates the files an AI agent needs to be useful: instructions, MCP config, VS Code settings, and evals.

Why

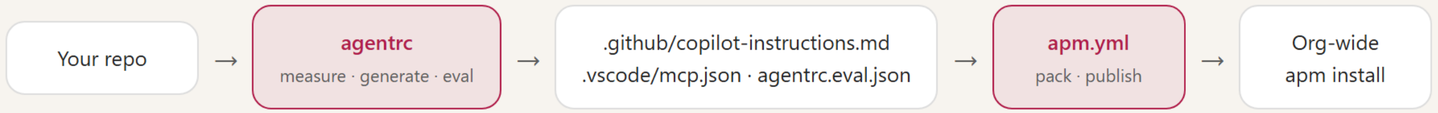
- Most repos ship **no** context for agents
- Hand-written instructions go stale as code evolves
- Without evals, you can't tell if context actually helps

```
$ npx github:microsoft/agentrc readiness
Scoring 9 pillars · 5-level maturity model

  Build & test          L4
  Linting               L2
  Architecture docs     L1
  MCP configuration     L0
  ...

$ npx github:microsoft/agentrc instructions
→ reading repo, generating .github/copilot-instructions.md
→ generating .vscode/mcp.json
→ generating agentrc.eval.json
✓ done

$ npx github:microsoft/agentrc eval
→ running 12 evals against generated instructions
✓ 11 passed, 1 regressed
```

AGENTRC — PART 2 OF 2

How AgentRC + APM compose

Generate locally with AgentRC. Distribute and version with APM.



IN YOUR PROJECT

`agentrc init` generates your baseline, then `apm install org/standards` pulls in shared org packages on top.

FOR YOUR TEAM

Package your best instructions/skills as an APM package; teammates get them with one `apm install`.

AT SCALE

`apm audit` + `apm-policy.yml` enforce standards; `agentrc eval` in CI catches context drift.

The `.instructions.md` format is shared by both tools — no conversion when moving content from AgentRC into an APM package.

ADOPTION PATH

How to start on Monday

STEP 1 · TRY IT

`curl -sSL https://aka.ms/apm-unix | sh`
then `apm install microsoft/apm-sample-package` on a throwaway repo.

STEP 2 · MEASURE YOUR REPO

`npx github:microsoft/agentrc readiness` — gives you a maturity score and a list of missing context to add first.

STEP 3 · PIN WHAT YOU HAVE

Move your existing `copilot-instructions.md` + skills into an `apm.yml`. Commit the lockfile.

STEP 4 · SHARE ACROSS THE TEAM

Extract the org-wide bits into a shared APM package. Teammates run `apm install`.

STEP 5 · GATE IN CI

Add `apm-action` + `agentrc readiness --fail-level 3` to PR checks. Stop drift.

STEP 6 · GOVERN

Roll out an `apm-policy.yml` at the org level. Tighten as you learn.

WRAP-UP

Three things to take away

- **Agent context is code.** Version it, pin it, ship it like dependencies.
- **APM gives you the manifest.** AgentRC gives you the content. Together they close the loop.
- **Security & governance are not afterthoughts.** Hashes, audits, policy — on every install, by default.

LINKS

github.com/microsoft/apm
microsoft.github.io/apm
github.com/microsoft/agentrc
agents.md · modelcontextprotocol.io

Q & A

Things I'd love to discuss: *what's the first APM package your team would build? Which MCP servers belong in your org allow-list? Where would AgentRC's eval loop fit in your CI today?*

THANK YOU

Questions?

Press ? for shortcuts · T for theme · ← → to navigate

Soham Dasgupta

Sr. Partner Solution Architect

Scan for portfolio